



华南理工大学
South China University of Technology

《高级语言程序设计大作业》

设计报告书

题目：电竞俱乐部管理系统

学 院	计算机科学与工程学院
专 业	计算机类
学生姓名	杨华城
学生学号	202430451747
指导老师	徐红云
课程编号	045101571
课程学分	2.0
起始日期	2025 年 3 月 1 日至 2025 年 6 月 12 日

电竞俱乐部管理系统

一 选题背景

随着电子竞技产业的蓬勃发展，全球电子竞技市场规模已突破千亿美元，中国电竞用户规模超 5 亿人，职业联赛、俱乐部运营、赛事商业化等体系日趋成熟。电竞行业从“小众娱乐”发展为“全民文化现象”，带动了游戏开发、直播、周边产业等完整生态链。在年轻人群体中，电子竞技已经成为很多人生活中不可或缺的一部分。从玩竞技游戏，到观看比赛直播，电竞这个词已深入人心，成为年轻人社交的重要一环。

本项目旨在设计并实现一个电子竞技俱乐部管理系统。在当前的电子竞技热潮下，一个俱乐部需要系统化地管理其财务、人员（教练和队员）、训练、比赛以及日常运营。本系统旨在模拟这一过程，为用户提供一个虚拟的俱乐部管理平台，使其能够体验俱乐部从创建、发展到参与竞赛的全过程。主要解决的问题包括：俱乐部信息的记录与更新、俱乐部资金的收支管理、队员和教练的招募与管理、俱乐部整体实力的评估、以及参与并记录锦标赛成绩等。

本系统使用面对对象的程序设计方法，参考并使用了 MVC（即 Model-View-Controller）的设计模式，通过将数据、视图和控制逻辑分离，实现系统的模块化和可维护性。此外，还使用了 `cmake` 进行项目管理，通过将源文件和头文件进行合理的组织，提高了代码的可读性和可维护性。在数据管理方面，使用了原生 C++ 的文件操作，通过读取和写入二进制文件来实现数据的持久化。代码中还运用到了 STL 容器、`random` 库等关键技术。

本设计的指导思想是构建一个面向对象的、模拟经营类的电子竞技俱乐部管理程序。通过将现实世界中的俱乐部、选手、教练、赛事等元素抽象为程序中的类和对象，模拟它们之间的交互和俱乐部的运营流程。系统应注重用户体验的流畅性和功能的完整性，让用户能够沉浸式地管理自己的电竞帝国。

1.1 人员分工

- 李睿：后台开发
- 杨华城：客户端开发、项目结构构建
- DAVUDOV SERDAR：界面美化

二 方案论证（设计理念）

本系统使用了类似 MVC（即 Model-View-Controller）的设计模式，并在设计类时充分考虑了接口的使用、多态性的使用等。最后在页面跳转协调和日志生成方面，使用了枚举表示状态。下面具体进行阐释。

2.1 MVC 设计模式

MVC 设计模式使前后端分离。视图层只需处理交互，而不需要进行具体的业务逻辑处理；控制器层和模型层则负责处理视图层的请求，进行业务逻辑处理。。

- **模型层（Model）** 负责封装应用的核心数据和业务逻辑。在本项目中，Club、Staff（及其派生类 Coach、Player）、Tournament、Log 类构成了模型层。它们不仅包含各自的属性，还定义了相关的操作方法（如资金变动、人员增删、战力计算等）。总体而言，模型层是系统中的最直接数据，并为 controllers 的业务操作提供相应的业务函数。
- **视图层（View）** 负责展示模型层的数据，以及用户与系统交互的界面。在本项目中，WelcomeView、MainMenuView、ClubInfoView、MarketView、TournamentView 等类属于视图层。它们接收用户的输入，并将模型数据显示给用户。视图层不直接处理业务逻辑，而是将用户操作传递给控制器。
- **控制器层（Controller）** 作为模型和视图之间的协调者。ClubController、MarketController、TournamentController 等类是控制器层。它们接收来自视图的用户请求，调用模型层相应的方法处理业务逻辑，并进行数据的更新。

2.2 类的设计

在对每一个类进行设计时，我们都尽量考虑接口以及抽象类的使用，以降低代码的耦合度，提高代码的复用性以及可维护性并使得代码逻辑更加清晰。

如在 view 层中，我们让其中的每一个类都继承于 IView 接口，然后在 main 函数中，利用多态性使用指向 IView 类的指针实现对不同 view 的调用，这体现了依赖倒置原则和接口隔离原则。

```
1 #pragma once
2
3 #include <iostream>
4 #include "ViewState.h"
5
```

```
6 class IView {
7 public:
8     virtual ViewState run() = 0;    //
        运行视图，返回下一级视图状态
9     virtual void initCtrl() = 0;    // 初始化控制器
10 };
```

Listing 1: IView 接口定义

而在 secret 板块，我们设计了 ISecret 接口，让需要接入 secret 的 Club 类和 Manager 类继承，随后在 secret.cpp 中定义 secret 相关的函数，在进行相关密码的操作时进行调用，避免重复使用相同的代码。

```
1 #pragma once
2 #include <string>
3
4 class ISecret {
5 public:
6     virtual std::string getSecret() = 0;
7     virtual void setSecret(std::string secret) = 0;
8 };
```

Listing 2: ISecret 接口定义

```
1 #pragma once
2
3 #include <string>
4 #include "ISecret.h"
5
6 // 模拟密钥输入
7 void Input(std::string &secret);
8
9 // 验证密钥
10 bool Verify(ISecret *sub, std::string secret);
11
12 // 修改密钥
```

```
13 void Change(ISecret *sub);
```

Listing 3: secret 相关函数声明

2.3 枚举表示状态

在页面跳转协调和日志生成的实现上，使用了枚举表示状态。在页面方面，我们定义了 ViewState 枚举类型，用于表示不同的页面状态。

```
1 #pragma once
2
3 enum class ViewState {
4     WelcomeView ,
5     CreateClub ,
6     LoadClub ,
7     InitMainMenu ,
8     MainMenuView ,
9     ClubInfoView ,
10    ClubLogView ,
11    MarketView ,
12    TournamentView ,
13    RankView ,
14    Exit ,
15    BackStageMenuView ,
16    BackStageStaffView ,
17    BackStageTourView
18 };
```

Listing 4: ViewState 枚举定义

在日志生成方面，我们定义了 LogOperation 枚举类型，用于表示针对不同的操作生成不同的日志。

```
1 #pragma once
2
3 enum LogOperation {
4     ParticipateTournament ,
```

```
5     SellStaff ,  
6     BuyStaff  
7 };
```

Listing 5: LogOperation 枚举定义

这样做的好处是，当需要添加新的操作或界面时，只需要在枚举类型中添加即可，并添加对应的类或函数，而不需要修改更加底层的代码，提高了可拓展性。同时，通过枚举类型，我们能够方便地判断当前的页面状态，从而对代码进行理解，提高了代码的可读性。

2.4 cmake 项目管理

由于采用了多文件的项目结构，并且为模型层、视图层、控制器层和 `include` 层分别设置了不同的文件夹，因此，我们使用 `cmake` 进行项目管理，让不同路径的源文件或头文件能够被正确地编译和链接。

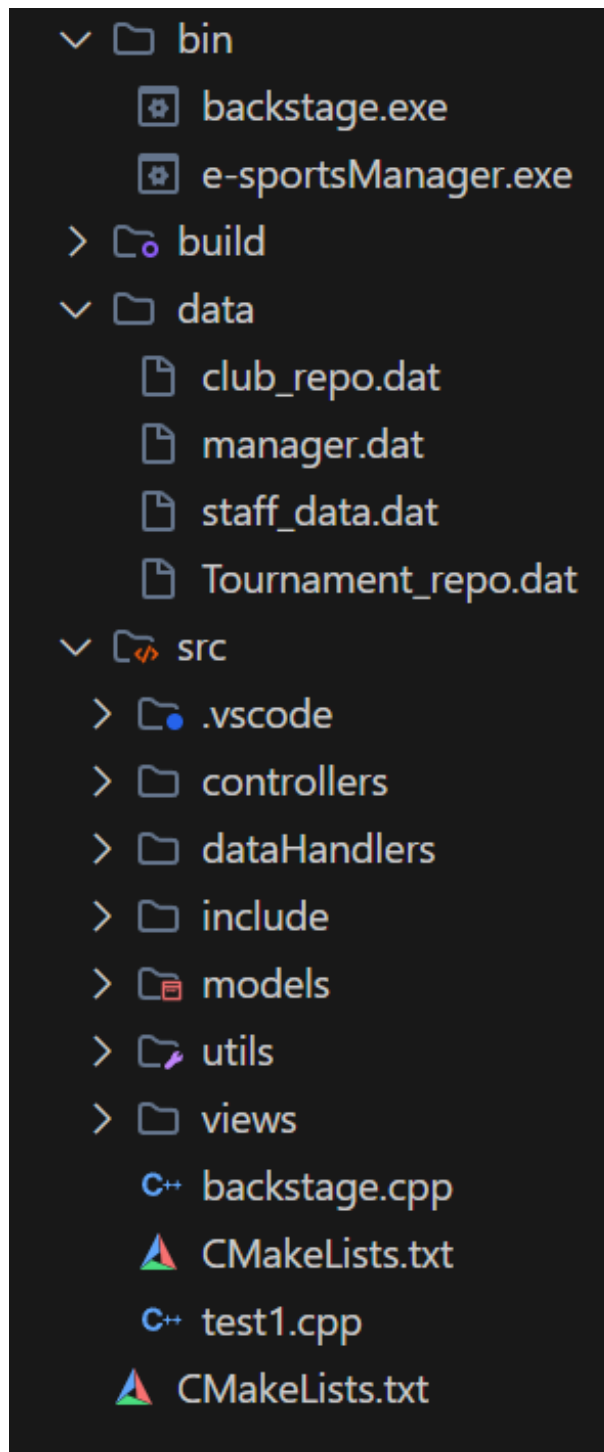


图 1: 项目文件结构

另外，cmake 的使用为后续的项目管理提供了便利，通过 cmake，我们能够方便地链接第三方库，如链接 qt 库进行 GUI 的开发。同时，cmake 的使用也提高了项目的可移植性，经过测试，我们的系统在 linux 系统上也能够正常运行。

2.5 实现的功能

本系统设计了客户端和后台管理两部分。后台功能包括创建新的选手并添加到市场上，以及赛事的创建，同时还提供了删除功能。客户端是主要的功能分布部分，下面根据页面进行分述。

1. 欢迎界面

- (a) 新建俱乐部并载入文件中
- (b) 从文件存档中加载俱乐部
- (c) 删除俱乐部
- (d) 密码验证，新建俱乐部时需创建密码，加载俱乐部时需验证密码

2. 俱乐部信息界面

- (a) 显示俱乐部信息
- (b) 显示俱乐部成员信息
- (c) 显示俱乐部动态
- (d) 修改密码

3. 市场界面

- (a) 显示市场信息
- (b) 买入选手
- (c) 卖出选手

4. 赛事界面

- (a) 显示赛事信息
- (b) 参与赛事
- (c) 根据比赛模拟算法计算出比赛结果

5. 排名界面

- (a) 按积分显示排名
- (b) 按战力显示排名
- (c) 按资金显示排名

2.6 系统安全性

1. 用户选择俱乐部时，需要输入密码，确保俱乐部的安全性
2. 用户修改俱乐部密码时，需要两次输入原密码
3. 在进入后台时，需要输入管理员密码
4. 在用户进行选择时，通过输入验证保证程序不会崩溃
5. 在用户进行市场交易或比赛时，会对俱乐部的资金进行判断，保证俱乐部的资金不会为负

2.7 数据的完整性

通过二进制文件存档的方式，保证了数据的完整性。在启动系统时，会自动从数据文件中对俱乐部数据、职员数据以及赛事数据进行读取；在退出系统时，会自动保存数据到数据文件中，保证了数据的完整性。

2.8 应用的运行环境及对应要求

2.8.1 运行环境

系统通过原生 c++ 进行开发，并通过终端进行输出，在 windows 系统上具有最佳的运行效果。在 linux 和 mac 上需要重新使用 cmake 进行 build，生成可执行文件后，即可运行。

2.8.2 对于要求

1. windows：在 windows 上需要保证系统有 GCC/G++ 编译器
2. linux、mac：由于需要重新使用 cmake 进行 build，因此需要保证系统有 cmake

三 过程论述

3.1 宏观框架

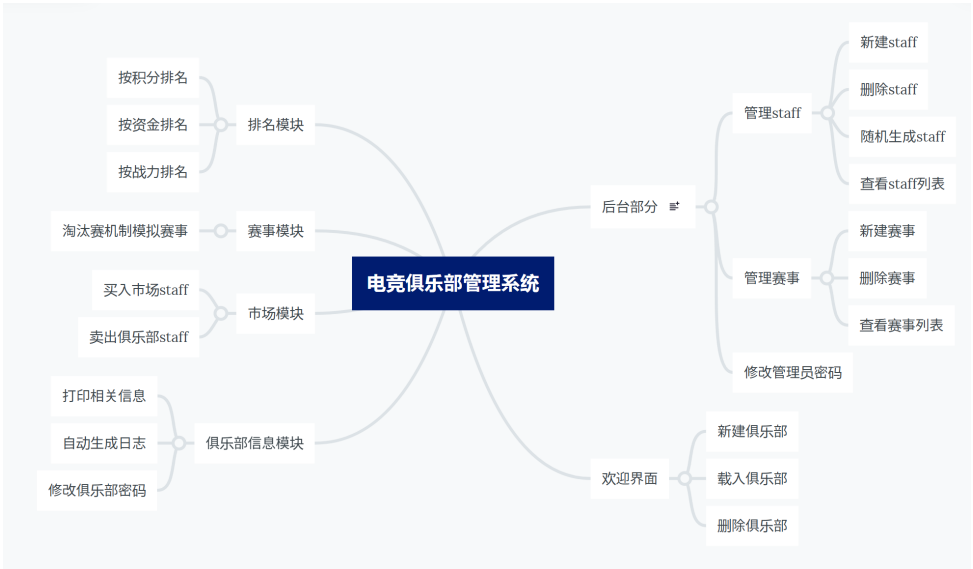


图 2: 宏观框架

3.2 欢迎界面

3.2.1 主界面

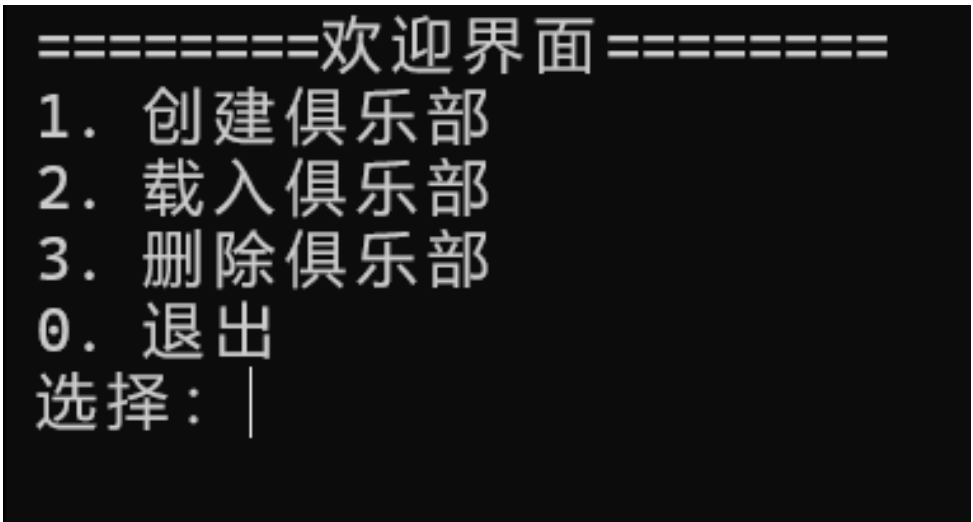


图 3: 欢迎界面

在欢迎界面中，使用“=”进行页面状态的提示，让用户直接输入数字进行选择，直接了当。

3.2.2 新建俱乐部

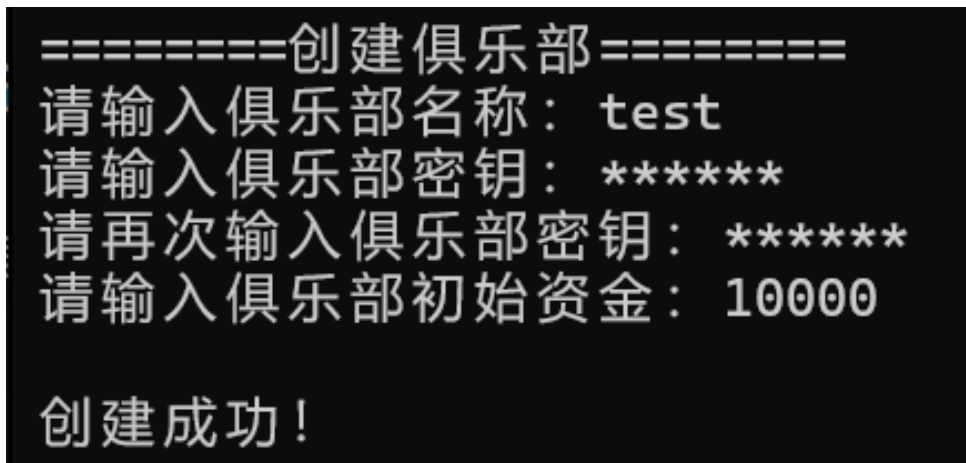


图 4: 新建俱乐部

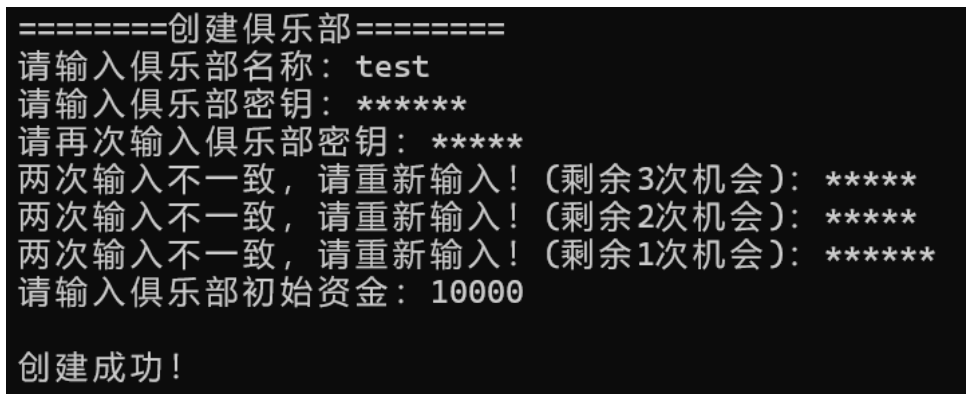


图 5: 新建俱乐部 (错误密码)

新建俱乐部在代码中的体现就是创建一个 `Club` 对象, 并将其添加到 `ClubRepo` 中。考虑到 MVC 模式, view 层中通过 `ClubController` 进行与 model 层的交互; 而 `ClubController` 则定义了 `std::shared_ptr<BinaryClubDataRepo>` 的私用成员进行与数据层的交互。

在输入密码时, 使用了密码隐藏的方式, 模拟真实的密码输入, 并要求用户两次输入密码, 保证能够准确记忆密码。以下是具体实现的代码, 同样后面在涉及密码输入时, 都采用了相同的方式, 便不再赘述。

```

1  std::string secret1, secret2;
2  // 模拟密钥输入
3  secret1.clear();
4  char ch;
5  while (true) {
6      ch = _getch();
  
```

```
7     if (ch == '\r' || ch == '\n') {
8         break;
9     } else if (ch == '\b') {
10        if (!secret1.empty()) {
11            secret1.pop_back();
12            std::cout << "\b \b";
13        }
14    } else {
15        secret1 += ch;
16        std::cout << '*';
17    }
18 }
19
20 int attempt = 4;
21 std::cout << "\n请再次输入俱乐部密钥：";
22 do {
23     // 模拟密钥输入
24     secret2.clear();
25     while (true) {
26         ch = _getch();
27         if (ch == '\r' || ch == '\n') {
28             break;
29         } else if (ch == '\b') {
30             if (!secret2.empty()) {
31                 secret2.pop_back();
32                 std::cout << "\b \b";
33             }
34         } else {
35             secret2 += ch;
36             std::cout << '*';
37         }
38     }
```

```
39
40     if (secret1 == secret2) break;
41
42     attempt--;
43     if (attempt == 0) break;
44     std::cout << "\n两次输入不一致，请重新输入！（剩余” <<
        attempt << ”次机会）：”；
45 } while (true);
46
47 if (attempt == 0) {
48     std::cout << "\n创建失败！\n”；
49     std::cout << std::endl;
50     return false;
51 }
```

Listing 6: 密码隐藏实现

3.2.3 载入俱乐部

```
=====俱乐部列表=====
1. tyloo
2. faze
3. falcons
4. navi
5. yhc
6. test
选择：6

请输入密钥：*****
密码正确！
```

图 6: 载入俱乐部

载入俱乐部在代码中的体现就是从 `ClubRepo` 中读取俱乐部数据,并显示在欢迎界面中,再让用户选择俱乐部,把选中的俱乐部设置为当前俱乐部,即将`ClubController`中的`currentClub*`指向选中的俱乐部,随即进行密码验证。

3.2.4 删除俱乐部

```
=====俱乐部列表=====
1. tyloo
2. faze
3. falcons
4. navi
5. yhc
6. test
选择： 6

删除成功！
```

图 7: 删除俱乐部

删除俱乐部在`ClubController`中的操作则与载入俱乐部相反,

3.3 主菜单

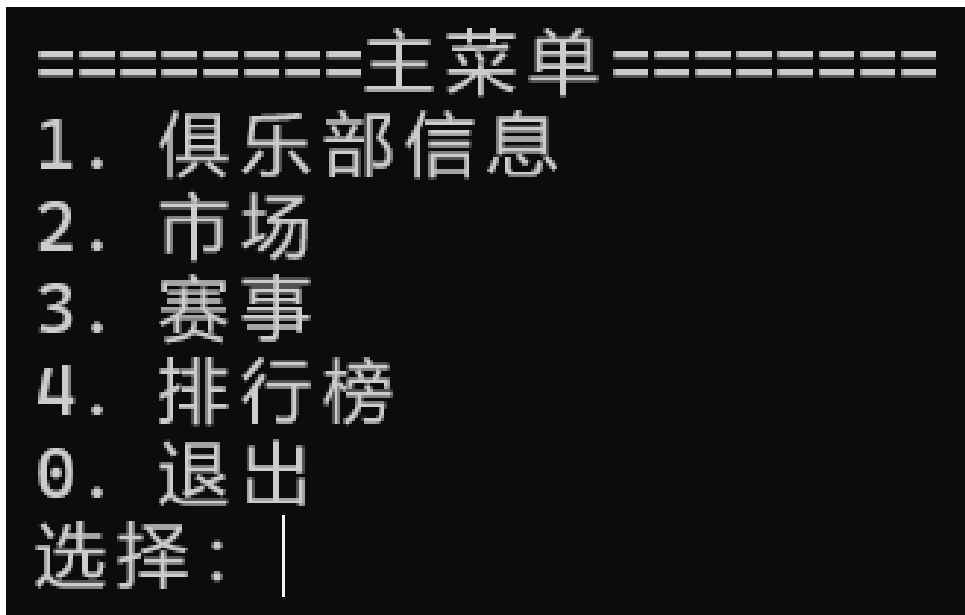


图 8: 主菜单

主菜单负责导航功能，下面给出页面导航的实现代码并进行说明。

代码为整个客户端的 `main` 函数，首先通过构造函数建立起数据仓库和相应控制器连接

```
1 // 数据仓库
2 auto clubRepo = std::make_shared<BinaryClubDataRepo>();
3 auto staffRepo = std::make_shared<BinaryStaffDataRepo>();
4 auto tourRepo = std::make_shared<BinaryTournamentDataRepo>();
5
6 clubRepo->load();
7 staffRepo->load();
8 tourRepo->load();
9
10
11 // 控制器
12 auto clubCtrl = new ClubController(clubRepo, staffRepo);
13 auto marketCtrl = new MarketController(staffRepo, clubRepo);
14 auto tourCtrl = new TournamentController(tourRepo, clubRepo);
```

Listing 7: 连接

然后在视图上定义了一个无序 `map` 容器，用于存储不同状态的视图，还定义了 `ViewState` 类型的 `state` 变量，用于存储下一状态

```

1 // 视图
2 std::unordered_map<ViewState, std::unique_ptr<IView>> views;
3
4 views[ ViewState::WelcomeView ] =
    std::make_unique<WelcomeView>(clubCtrl);
5 views[ ViewState::MainMenuView ] =
    std::make_unique<MainMenuView>(clubCtrl, marketCtrl,
    tourCtrl);
6 views[ ViewState::ClubInfoView ] =
    std::make_unique<ClubInfoView>(clubCtrl);
7 views[ ViewState::ClubLogView ] =
    std::make_unique<ClubLogView>(clubCtrl);
8 views[ ViewState::MarketView ] =
    std::make_unique<MarketView>(marketCtrl);
9 views[ ViewState::TournamentView ] =
    std::make_unique<TournamentView>(tourCtrl);
10 views[ ViewState::RankView ] =
    std::make_unique<RankView>(clubCtrl);
11
12 ViewState state = ViewState::WelcomeView;

```

Listing 8: 视图定义

最后只需在主循环中，通过 `state` 变量的值，调用 `views` 中对应状态的视图的 `run` 方法，即可实现页面跳转，在 `state == ViewState::Exit` 时，退出循环，结束程序。

```

1 // 程序主循环
2 // 程序主循环
3 while (state != ViewState::Exit) {
4     if (state == ViewState::InitMainMenu) {
5         views[ ViewState::MainMenuView ]->initCtrl();
6         state = ViewState::MainMenuView;
7     }

```



```

8   state = views[state]->run();
9 }

```

Listing 9: 主循环

3.4 俱乐部信息

```

=====俱乐部信息=====
俱乐部名称      tyloo
资金            3850
积分            7
俱乐部战力      5743

-----成员列表-----
职务      名字      战力      价格
-----
教练      xalo      132      660
选手      seli      287      1435
选手      nuqeqi    349      1745
选手      vi        562      2810

1. 动态
2. 修改密钥
0. 返回
选择: |

```

图 9: 俱乐部信息

首先对当前俱乐部相关信息进行打印。

3.4.1 日志（动态）

```

=====俱乐部动态=====
2025-06-07 23:06:36   买入 xalo      -660      9340      |      +0      0
2025-06-07 23:06:41   买入 seli      -1435     7905      |      +0      0
2025-06-07 23:06:43   买入 nuqeqi    -1745     6160      |      +0      0
2025-06-07 23:06:45   买入 vi        -2810     3350      |      +0      0
2025-06-07 23:06:47   买入 sepo      -2170     1180      |      +0      0
2025-06-07 23:19:50   参加 AustinMajor +400      1580      |      +5      5
2025-06-08 14:27:19   卖出 sepo      +2170     3750      |      +0      5
2025-06-08 14:27:52   参加 AustinMajor +100      3850      |      +2      7

0. 返回
选择: |

```

图 10: 日志

一条 log 的构成为：时间 + 事件 + 资金变化 + 当前资金 + 积分变化 + 当前积分。实际上作为一个 `std::string` 对象添加到当前俱乐部的 `logs` 中。下面具体说明 log 的生成代码。

首先是Log类的定义

```

1 class Log {
2     std::string log;
3     std::string staff_name;
4     std::string tournament_name;
5     public:
6         // 设置选手
7         void setStaff(std::string staff) {this->staff_name =
            staff;}
8
9         // 设置赛事
10        void setTournament(std::string tournament)
            {this->tournament_name = tournament;}
11
12        // 生成日志
13        void generateLog(LogOperation operation, int
            fund_change, int points_change, int fund, int
            points);
14
15        // 获取日志
16        std::string getLog() const {return log;}
17
18 };

```

Listing 10: Log 类定义

重点看generateLog函数的实现。首先通过ctime库的time函数获取当前时间,然后通过localtime函数获取tm*格式的实际以便进行格式化,最后通过std::stringstream加入到log中

```

1 // 设置时间
2 auto now = std::time(nullptr);
3 auto local_time = std::localtime(&now);
4
5 std::stringstream ss;

```

```

6 ss << std::put_time(local_time, "%Y-%m-%d %H:%M:%S");
7 this->log += ss.str();
8 this->log += std::string(25 - ss.str().length(), ' ');

```

Listing 11: 时间戳生成

然后是具体的日志生成，由于不同操作对应的日志生成机制类似，这里就仅展示参加赛事操作的具体代码。fix 用于消除 int 为负带来的格式影响。

```

1 int fix = 0;
2 // 生成日志
3 switch (operation) {
4     // 参加赛事
5     case LogOperation::ParticipateTournament:
6         this->log += "参加" + tournament_name +
7             std::string(15 - tournament_name.length(), ' ');
8         fix = 0;
9         if (fund_change >= 0) {this->log += "+"; fix = 1;}
10
11        this->log += std::to_string(fund_change) +
12            std::string(10 -
13                std::to_string(fund_change).length() - fix, ' ')
14            + std::to_string(fund) + std::string(10 -
15                std::to_string(fund).length(), ' ')
16            + '|' + std::string(5, ' ');
17
18        fix = 0;
19        if (points_change >= 0) {this->log += "+"; fix = 1;}
20
21        this->log += std::to_string(points_change) +
22            std::string(10 -
23                std::to_string(points_change).length() - fix, ' ')
24            + std::to_string(points) +

```

```

21         std::string(10 -
22         std::to_string(points).length(), '
        ');
        break;
    }

```

Listing 12: 日志生成

在TournamentController中，调用Log的相关函数函数进行具体的日志生成，并添加到当前俱乐部中

```

1 // 生成日志
2 Log log;
3 log.setTournament(current_tour->getName());
4 log.generateLog(LogOperation::ParticipateTournament,
    fund_bonus[i] - current_tour->getEntryFee(),
    points_bonus[i], club->getFund(), club->getPoints());
5 club->addLog(log.getLog());

```

Listing 13: 日志添加

3.5 市场

3.5.1 购买选手

在市场主界面中会展示当前的市场选手

=====市场=====				
编号	名字	职位	能力	价格
1.	ki	选手	257	1285
1. 买入				
2. 出售				
0. 返回				
选择:				

图 11: 市场

进行买入操作后选手会被添加到当前俱乐部的选手列表中，并从市场中移除。

```

输入要买入选手编号： 1

交易成功！

=====市场=====
编号      名字      职位      能力      价格
-----
1. 买入
2. 出售
0. 返回
选择：|

```

图 12: 购买选手

购买选手操作与MarketController有关，下面将根据代码进行分析。

首先在构造MarketController时，会从StaffRepo中读取所有state为false的选手，即未加入俱乐部的选手，并添加到marketStaff中，随后进行打印。

```

1 MarketController(std::shared_ptr<BinaryStaffDataRepo>
  staffRepo, std::shared_ptr<BinaryClubDataRepo> clubRepo) :
2   staff_repo(std::move(staffRepo)),
  club_repo(std::move(clubRepo)) {
3   // 市场选手
4   auto & repo = staff_repo->getRepo();
5   if (!repo.empty()) {
6       for (const auto & item : repo) {
7           if (!item->getState()) {
8               market_staff.push_back(item->getID());
9           }
10      }
11  }
12 }

```

Listing 14: MarketController 构造

接下来进行购买操作，其涉及的操作如下：

1. 扣除俱乐部资金，若资金不足则无法购买
2. 添加选手至俱乐部中并更新俱乐部战力

3. 在市场中删除该选手

4. 生成日志

```
1 // 处理买入操作
2 // 1. 扣除俱乐部资金
3 if (current_club->getFund() < target_staff->getPrice()) {
4     std::cout << "俱乐部资金不足! \n";
5     return false;
6 }
7 current_club->changeFund(-target_staff->getPrice());
8
9 // 2. 添加选手
10 if (auto target_coach = dynamic_cast<Coach*>(target_staff)) {
11     target_coach->changeState();
12     current_club->addCoach(target_coach->getID());
13 } else if (auto target_player =
14     dynamic_cast<Player*>(target_staff)) {
15     target_player->changeState();
16     current_club->addPlayer(target_player->getID());
17 }
18 // 3. 更新俱乐部战力
19 current_club->setPower(staff_repo);
20
21 // 4. 市场中删除该选手
22 market_staff.erase(it_target_staff);
23
24 // 5. 生成日志
25 Log log;
26 log.setStaff(target_staff->getName());
27 log.generateLog(LogOperation::BuyStaff,
28     -(target_staff->getPrice()), 0, current_club->getFund(),
29     current_club->getPoints());
```

```
28 current_club -> addLog(log.getLog());
```

Listing 15: 买入操作

3.5.2 卖出选手

选择卖出操作时，首先会将俱乐部选手列表打印，以供用户选择

```
选择：2

=====俱乐部选手=====
编号    职位    名字        能力    价格
-----
1.      教练    xalo        132     660
2.      选手    seli        287     1435
3.      选手    nuqeqi      349     1745
4.      选手    vi          562     2810
5.      选手    ki          257     1285

输入要卖出选手编号：5

交易成功！

=====市场=====
编号    名字    职位        能力    价格
-----
1.      ki      选手        257     1285

1. 买入
2. 出售
0. 返回
选择：|
```

图 13: 卖出选手

卖出选手操作与买入操作类似，首先设置可卖出选手列表，即俱乐部中所有选手，在代码中体现为遍历。然后进行与买入完全相反的操作即可。

3.6 赛事

进入赛事界面后同样会打印当前的赛事，用户可选择参加

```
=====赛事=====
序号    赛事名称    参赛队伍数    参赛费用    总奖金
-----
1       major      8             3000        34500
2       AustinMajor 4             100         1100

0. 返回
选择：|
```

图 14: 赛事

赛事部分最核心的是对赛事的模拟，我们设计了TournamentController类，并通过 random 库设置概率，下面是具体的实现。

考虑到参赛队伍数量的限制，首先需要对参赛队伍进行设置。先将当前俱乐部添加至参赛队伍中，然后定义一个indices，用于存储除当前俱乐部外其他俱乐部的 ID，将其随机打乱后，取出前 `tour->getTeamNum() - 1` 个俱乐部作为参赛队伍。

```

1 // 设置比赛队伍
2 void TournamentController::setGameClubs() {
3     // 设置参赛队伍
4     game_clubs.resize(current_tour->getTeamNum());
5     game_clubs[0] = current_club->getID();
6
7     // 随机选择参赛队伍
8     std::vector<int> clubs = club_repo->getRepoID();
9     std::vector<int> indices;
10
11     for (size_t i = 0; i < clubs.size(); i++) {
12         if (clubs[i] == current_club->getID()) {
13             continue;
14         }
15         indices.push_back(clubs[i]);
16     }
17     std::shuffle(indices.begin(), indices.end(), rng);
18
19     for (int i = 1; i < current_tour->getTeamNum(); i++) {
20         game_clubs[i] = indices[i - 1];
21     }
22 }

```

Listing 16: 参赛队伍设置

接下来是具体的比赛模拟，采用淘汰赛机制，参赛队伍两两间进行比赛，淘汰败者，直至冠军产生。这样的赛制要求比赛队伍数量必须为 2 的幂次。

两队比赛时，使用logistic函数计算胜率，战力高的一方胜率更高。

```

1 // 计算A队击败B队的概率

```



```

2 double TournamentController::calculateWinProbability(int a,
   int b) {
3     // 获取队伍
4     auto club_a = club_repo->getClub(a);
5     auto club_b = club_repo->getClub(b);
6
7     // 计算A队击败B队的概率（使用logistic函数）
8     double diff = club_a->getPower() - club_b->getPower();
9     double prob = 1 / (1 + exp(-diff));
10    return prob;
11 }
12
13 // 模拟两队比赛
14 int TournamentController::simulateGame(int a, int b) {
15     // 计算A队击败B队的概率
16     double prob = calculateWinProbability(a, b);
17
18     std::uniform_real_distribution<double> dist(0, 1);
19     double random = dist(rng);
20
21     return random < prob ? a : b;
22 }

```

Listing 17: 模拟两队比赛

匹配机制上，由于参赛队伍已进行随机打乱，因此无需再次打乱，直接两两配对即可，然后将败者排至末尾，将胜者排至前面。

```

1 // 模拟赛事
2 int round = 0;
3 int num = current_tour->getTeamNum();
4 while ((num /= 2) != 1) {
5     round++;
6 }
7 round++;

```

```

8
9 int team_num = current_tour->getTeamNum();
10 game_result = game_clubs;
11 for (int i = 0; i < round; i++) {
12     std::vector<int> team_round = game_result;
13     for (int j = 0; j < team_num; j += 2) {
14         int winner = simulateGame(team_round[j], team_round[j
15             + 1]);
16         game_result[j / 2] = winner;
17         game_result[j / 2 + team_num / 2] = team_round[j] +
18             team_round[j + 1] - winner;
19     }
20     team_num /= 2;
21 }
22 const std::vector<int> & fund_bonus =
23     current_tour->getFundBonus();
24 const std::vector<int> & points_bonus =
25     current_tour->getPointsBonus();
26
27 std::reverse(game_result.begin(), game_result.end());

```

Listing 18: 比赛模拟

最后, 根据比赛结果, 更新俱乐部资金和积分, 并为每支队伍生成日志, 并打印比赛结果。

```

1 // 按照名次分配奖励
2 for (int i = 0; i < current_tour->getTeamNum(); i++) {
3     auto club = club_repo->getClub(game_result[i]);
4     club->changeFund(fund_bonus[i]);
5     club->changePoints(points_bonus[i]);
6
7     // 生成日志
8     Log log;

```

```

9      log.setTournament(current_tour->getName());
10     log.generateLog(LogOperation::ParticipateTournament,
        fund_bonus[i] - current_tour->getEntryFee(),
        points_bonus[i], club->getFund(), club->getPoints());
11     club->addLog(log.getLog());
12 }

```

Listing 19: 比赛结果处理

```

=====赛事=====
序号   赛事名称       参赛队伍数   参赛费用   总奖金
-----
1      major          8           3000       34500
2      AustinMajor    4           100        1100

0. 返回
选择: 2
=====赛事结果=====
1-test
2-yhc
3-navi
4-tyloo

0. 返回
选择: |

```

图 15: 比赛结果

3.7 排名

排名部分，我们预设了积分、战力和资金三种排名方式供用户选择

```

=====排名=====
1. 按积分排序
2. 按战力排序
3. 按资金排序
0. 返回
选择: |

```

图 16: 排名

-----按积分排序-----

1-tyloo	8
2-falcons	7
3-faze	6
4-test	5
5-navi	4
6-yhc	3

=====排名=====

1. 按积分排序
2. 按战力排序
3. 按资金排序
0. 返回

选择： 2

-----按战力排序-----

1-tyloo	8660
2-navi	6244
3-falcons	5094
4-faze	4800
5-yhc	2452
6-test	0

=====排名=====

1. 按积分排序
2. 按战力排序
3. 按资金排序
0. 返回

选择： 3

排序时主要使用到了`algorithm`库中的`sort`函数，通过定义不同的`lambda`函数的实现不同属性的排序，下面仅给出战力排序的代码，其余类似

```

1 // 按战力排序
2 void ClubController::printRankByPower() {
3     auto & rank_clubs = club_repo->getRepo();
4
5     // 按战力进行排序
6     std::sort(rank_clubs.begin(), rank_clubs.end(),
7     [](const std::unique_ptr<Club> & club1, const
8         std::unique_ptr<Club> & club2) {
9         return club1->getPower() >= club2->getPower();
10     });
11
12     // 打印排名
13     int rank = 1;
14     for (const auto & club : rank_clubs) {
15         std::cout << rank << "—" << club->getName() <<
16             std::string(15 - club->getName().length(), ' ') <<
17             club->getPower() << std::endl;
18         rank++;
19     }
20 }

```

Listing 20: 排名

四 结果分析

系统成功实现了电子竞技俱乐部管理的核心功能，包括俱乐部的创建、登录、信息查看、资金管理、人员（教练和选手）的雇佣与解雇、市场交易以及参与锦标赛的框架。用户可以通过控制台界面与系统进行交互，完成一系列管理操作。

在数据持久化上，俱乐部、职员和赛事数据能够正确地通过二进制文件（.dat）进行加载和保存。这意味着用户的游戏进度可以被保留，下次启动程序时可以继续之前的状态。

MVC-like 的架构使得代码职责分明。例如，Club 类的修改主要影响数据层面，而 ClubInfoView 的修改主要影响界面展示，两者通过 ClubController 解耦。

系统通过一系列视图（WelcomeView, MainMenuView 等）和 ViewState 状态机来管理用户交互流程，提供了相对清晰的操作路径。

五 课程设计总结

5.1 遇到的问题解决方法

在系统构建初期，由于对智能指针的理解问题，在 main 函数中构造模型层、数据仓库、控制器层以及视图层的对象时一律使用了 shared_ptr，在成员函数中又使用 shared_ptr，进而导致了内存泄漏。通过对智能指针的进一步学习以及 debug，将模型层的对象置于各自的 repo 类中进行存储，并通过 ID 进行数据的访问；尽可能将 shared_ptr 改为 unique_ptr，仅保留 repo 类的 shared_ptr。

编码问题也带来了不少困扰。由于使用 vscode 作为编辑器，开始想使用 GBK 以支持中文，但依旧出现了中文乱码问题。在网上搜索资料后发现，问题出在 windows 的 powershell 默认使用 GB2312 编码，而 cmd 默认使用 UTF-8 编码。最后将系统的 powershell 编码和文件编码统一改为 UTF-8，解决了问题。

在优化 BinaryXXXRepo 时，我们考虑将整个需要进行存储的模型层类继承于 IData 类，然后只定义 BinaryDataRepo 这一统一数据仓库，其中使用 vector<unique_ptr<IData>> 存储数据。但后来发现，这一方案难以实现对不同模型层类的分别存储。后面考虑了使用模板类，但由于 Staff 类是抽象类，无法进行模板实例化，同时我们已经完成了其他部分的编写，为 Player 类和 Coach 类分别建立仓库也许需要变动很多，加之时间不足，就放弃了这一方案。

5.2 程序调试能力的思考

引入 cmake 进行项目管理后，程序的调试变得清晰便捷。通过 CMake: Set launch\debug target 将目标设置在想要的测试程序上，并设置好断点，便可以轻松进行调试，观察变量、指针的变化，便可以发现问题点，准确进行修正。

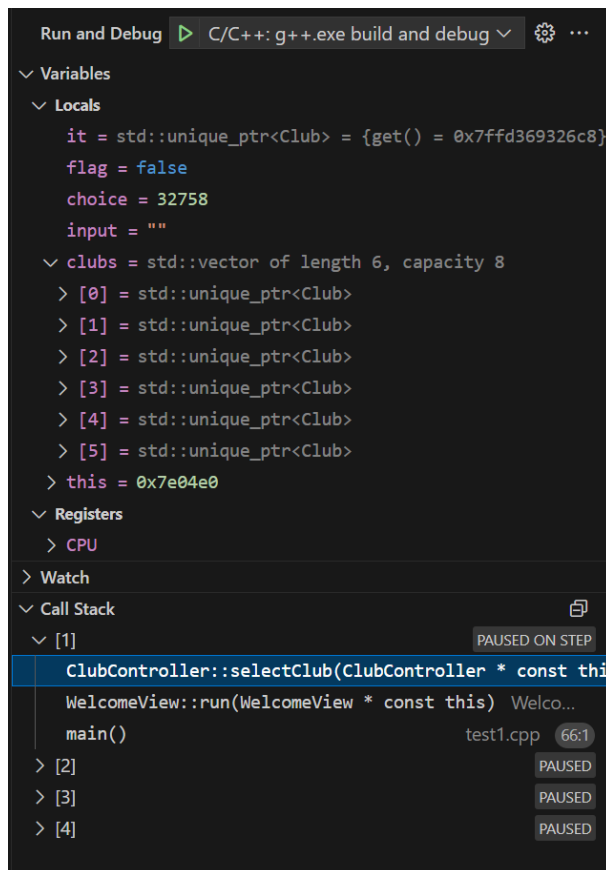


图 18: 调试

5.3 华为云资源的使用

通过华为云的资源，成功部署了服务器，并在服务器上成功运行了系统，体现了跨平台性。在部署中，遇到了conio.h头文件的问题，最后通过在 github 上寻找替代库，成功解决了问题。

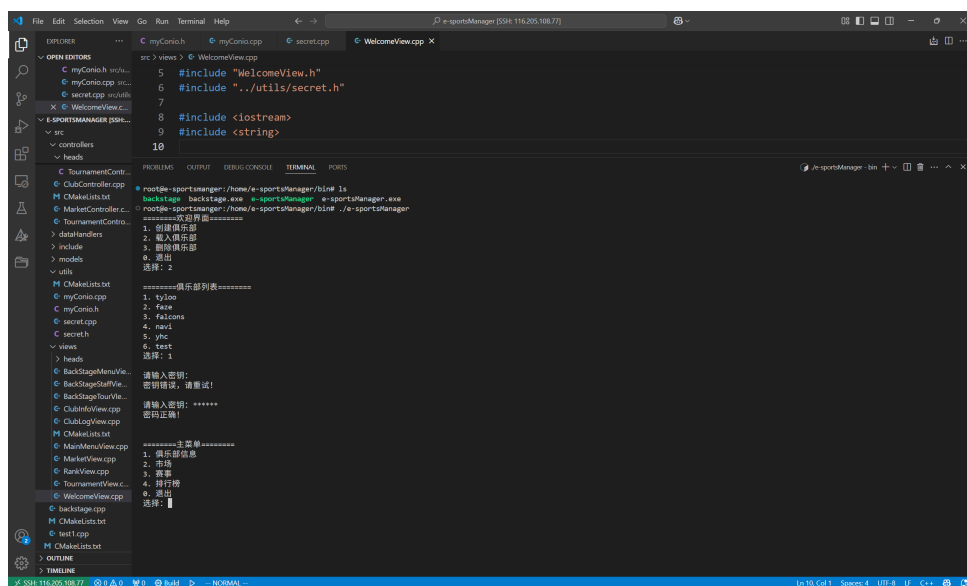


图 19: 服务器上部署

5.4 课程设计过程中的收获和体会

整个实现过程是一次宝贵的学习经历。从最初只有一个模糊的想法，到逐步设计数据结构、模块接口，再到一行行代码的编写和调试，最终看到系统能够顺利运行，带来了巨大的成就感。在这个过程中，MVC 设计模式以及视图状态机给了很大的启发

我体会到，清晰的设计是成功的一半，良好的代码规范和模块化思想能极大提高开发效率和后续维护的便利性。同时，也认识到软件开发往往是一个迭代的过程，不可能一蹴而就，需要不断地测试、发现问题、解决问题并进行优化。这次设计也让我对 C++ 语言的强大功能以及其在内存方面的使用有了更深的理解。